

ARM PrimeCell

SMC PL092 r1p3

User Guide

ARM PrimeCell SMC (PL092) User Guide

© Copyright ARM Limited 2003. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on this product

If you have any comments or suggestions about this product, please contact your supplier giving:

- The product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change History	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	DIRECTORY STRUCTURE	2
3.1	Peripheral's source code directory structure	3
3.2	Generic Synopsys synthesis reference directory structure	4
3.3	Generic Verification directory structure	5
3.4	Generic Integration Test directory structure	6
4	USAGE INSTRUCTIONS	7
4.1	Introduction	7
4.2	Design tools used	7
4.3	Reference Cell library	7
4.4	Peripheral Development Environment	8
4.4.1	Environment variables	9
4.4.2	Makefile commands	9
4.5	Running Design Tools	11
4.5.1	Generating the Functional Test Vectors (BusTalk)	11
4.5.2	Compiling the Peripheral RTL Code	11
4.5.3	Running Functional Simulations	12
4.5.4	Running Synthesis	12
4.5.4.1	scr_common directory	14
4.5.4.2	scripts directory	14
4.5.4.3	log directory	14
4.5.4.4	report directory	15
4.5.5	Running Netlist Simulations	16
4.5.6	Generating the Integration Test Vectors	17
4.5.7	Running the Integration Test RTL Simulation	18
4.5.8	Simulation error and warning messages	18
4.5.8.1	VHDL	18
4.5.8.2	Verilog	18
4.5.9	Use of Denali simulation models	19

1 ABOUT THIS DOCUMENT

1.1 Change History

Issue	Date	Change
A	21 April, 1999	First release, supersedes SCO15 USGD 0000 B.
B	1 July, 1999	Second release, amendment of minor errors.
C	7 July 2000	Applies to peripherals released after 1 July 2000.
D	15 September 2000	Feedback from Out of Box Experiences (OOB)
E	27 November 2001	PL092 specific version generated
F	4 Feb 2003	Updated for release PL092 r1p2
G	10 June 2003	Updated for release r1p3-00ltd0

1.2 References

For further information refer to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	PL092 DDES 0000	ARM PrimeCell (PL092) Design Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
IP	Intellectual Property
RTL	Register Transfer Level
SMC	Static Memory Controller

2 SCOPE

This document gives instructions for the installation and use of the ARM PrimeCell Static Memory Controller (SMC) PL092.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

The ARM PrimeCell SMC PL092 Design Manual [Ref.2] contains details of the rtl blocks, testbench and test code descriptions.

3 DIRECTORY STRUCTURE

The peripheral development environment is stored in two directory structures, a peripheral specific directory structure and a project global directory structure. This allows projects that contain multiple peripherals to use the same shared data. The top level directory structures are shown in **Figure 3-1**.

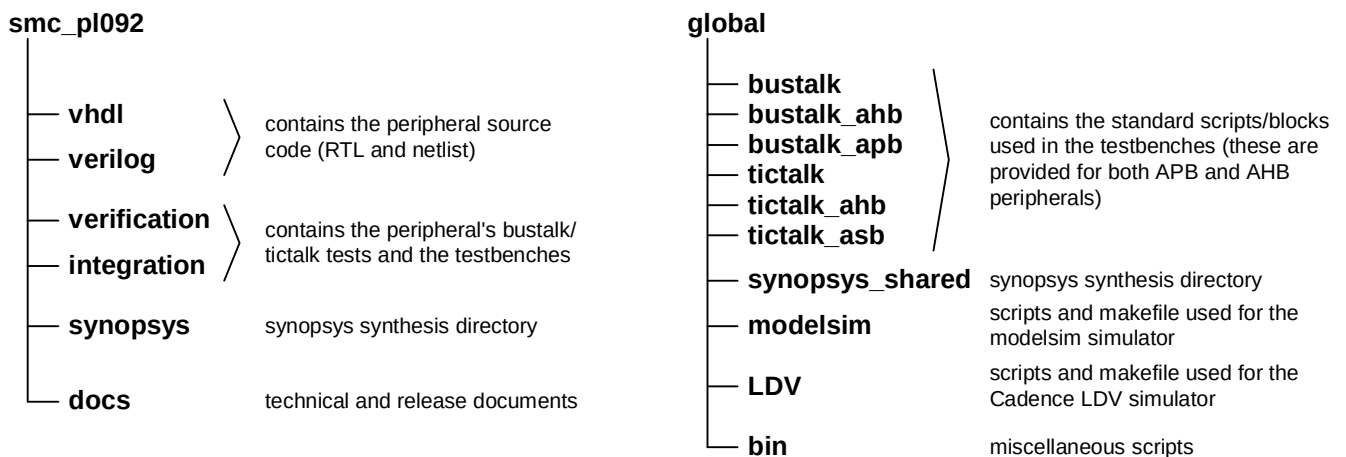


Figure 3-1. Peripheral Development Environment Directory Structure

3.1 Peripheral's source code directory structure

The peripheral's source code will be stored in two identical directory structures, as shown in **Figure 3-2**. The source code is in the "rtl_source" directory and the synthesised netlists and associated sdf will be placed in the netlist directory.

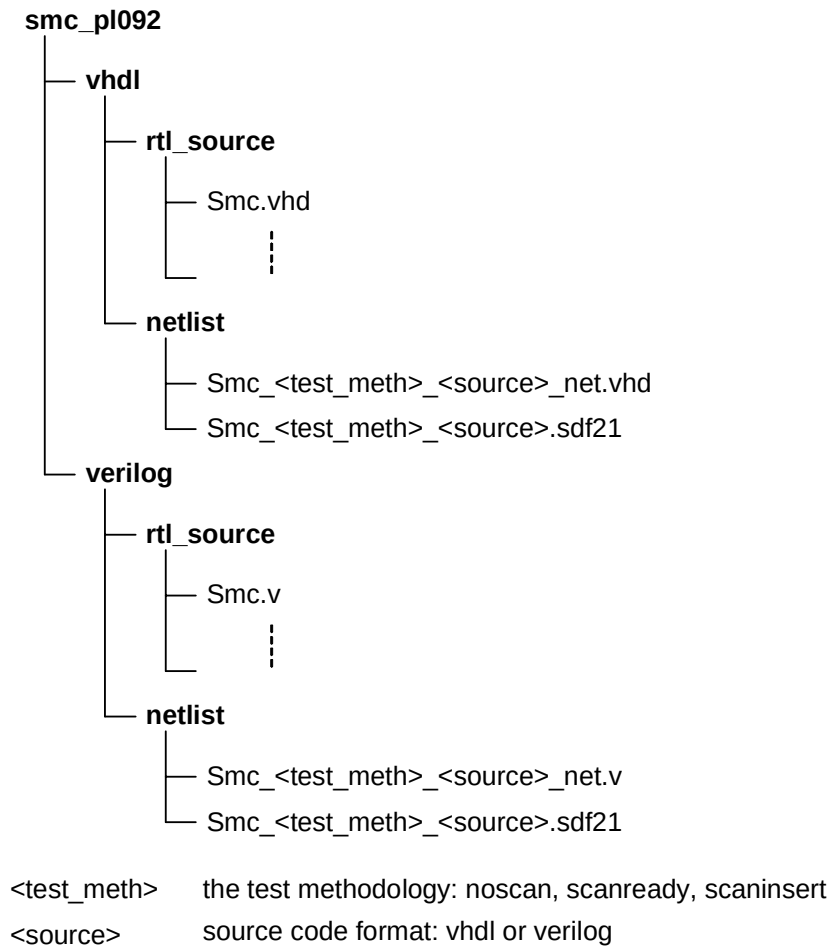


Figure 3-2. Source Code Directory Structure

3.2 Generic Synopsys synthesis reference directory structure

Synthesis is supported for Synopsys Design Compiler with the required command scripts and constraint files being supplied to the customer. The synthesis environment directory structure is shown in **Figure 3-3** Generic Synopsys synthesis reference directory structure.

The synthesised netlist is written into the source code area's netlist.

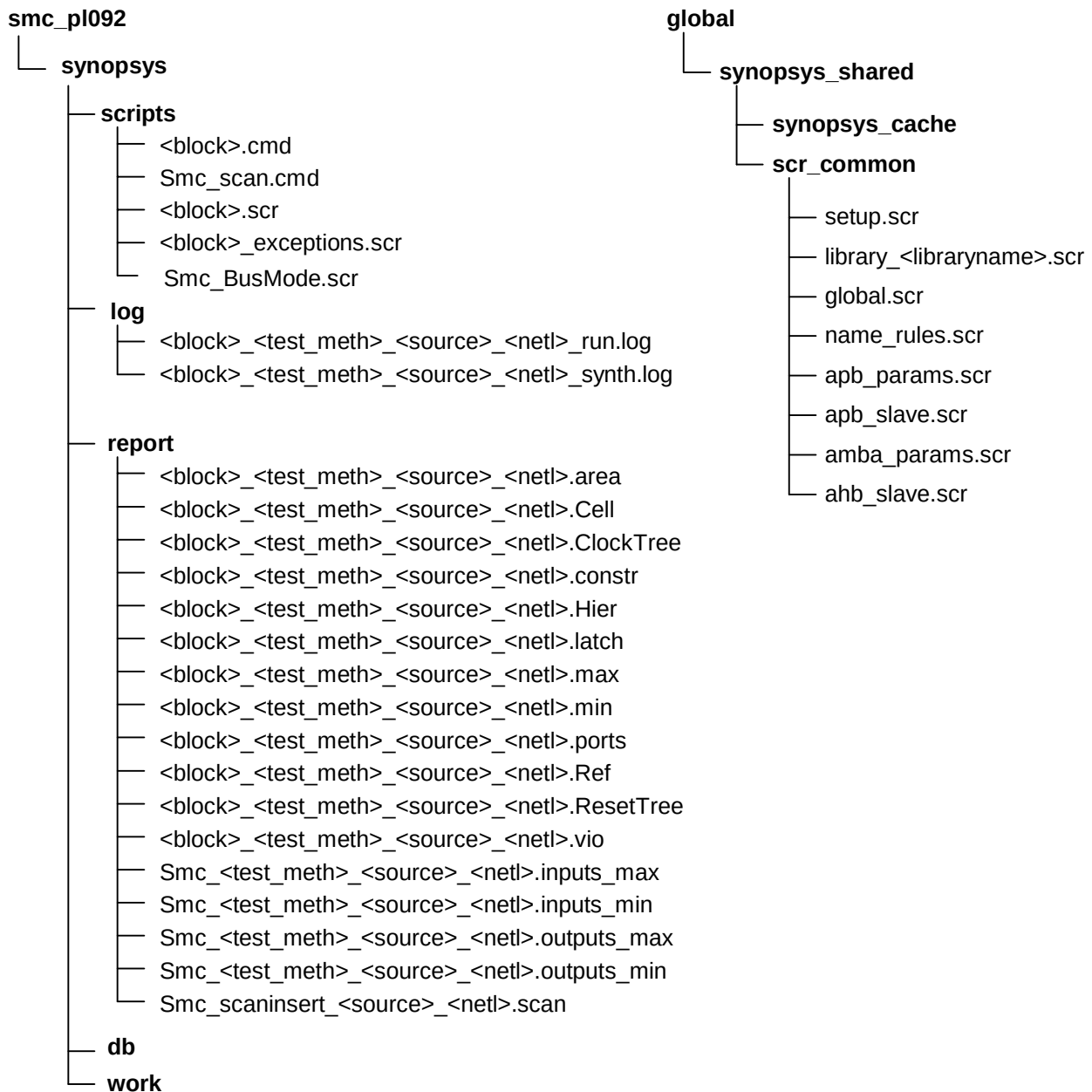


Figure 3-3. Generic Synopsys synthesis reference directory structure

3.3 Generic Verification directory structure

The verification directory contains the standalone verification testbenches in both VHDL and Verilog and the BusTalk test program, which is used for functional verification.

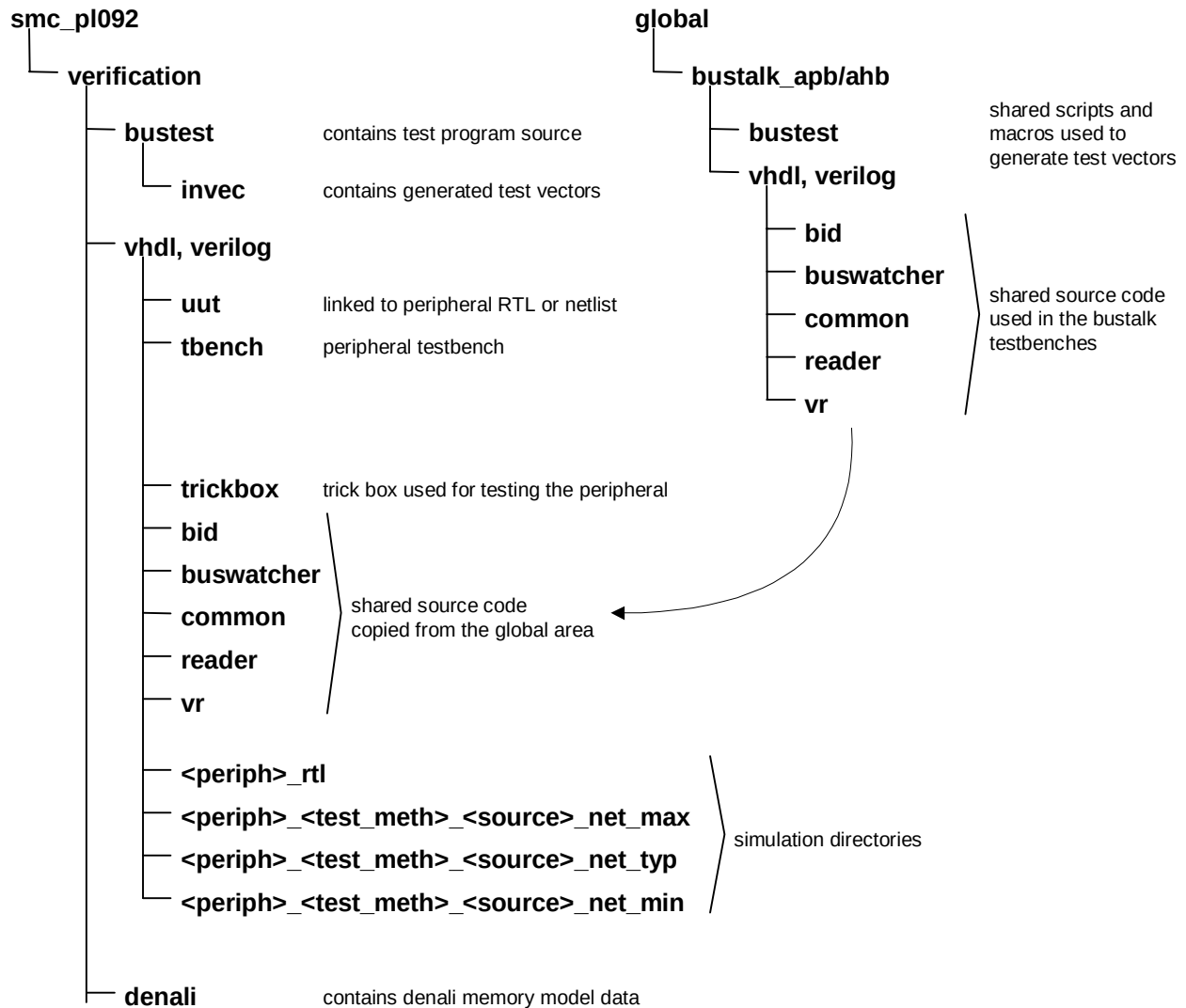


Figure 3-4. Generic Verification directory structure

The **uut** directory is linked either to the RTL source or the netlist directory depending on the simulation being performed.

3.4 Generic Integration Test directory structure

The integration test environment contains both Verilog and VHDL testbenches and the integration test. The testbenches are a minimal EASY system supporting TIC testing.

Figure 3-5 illustrates the generic integration test directory structure:

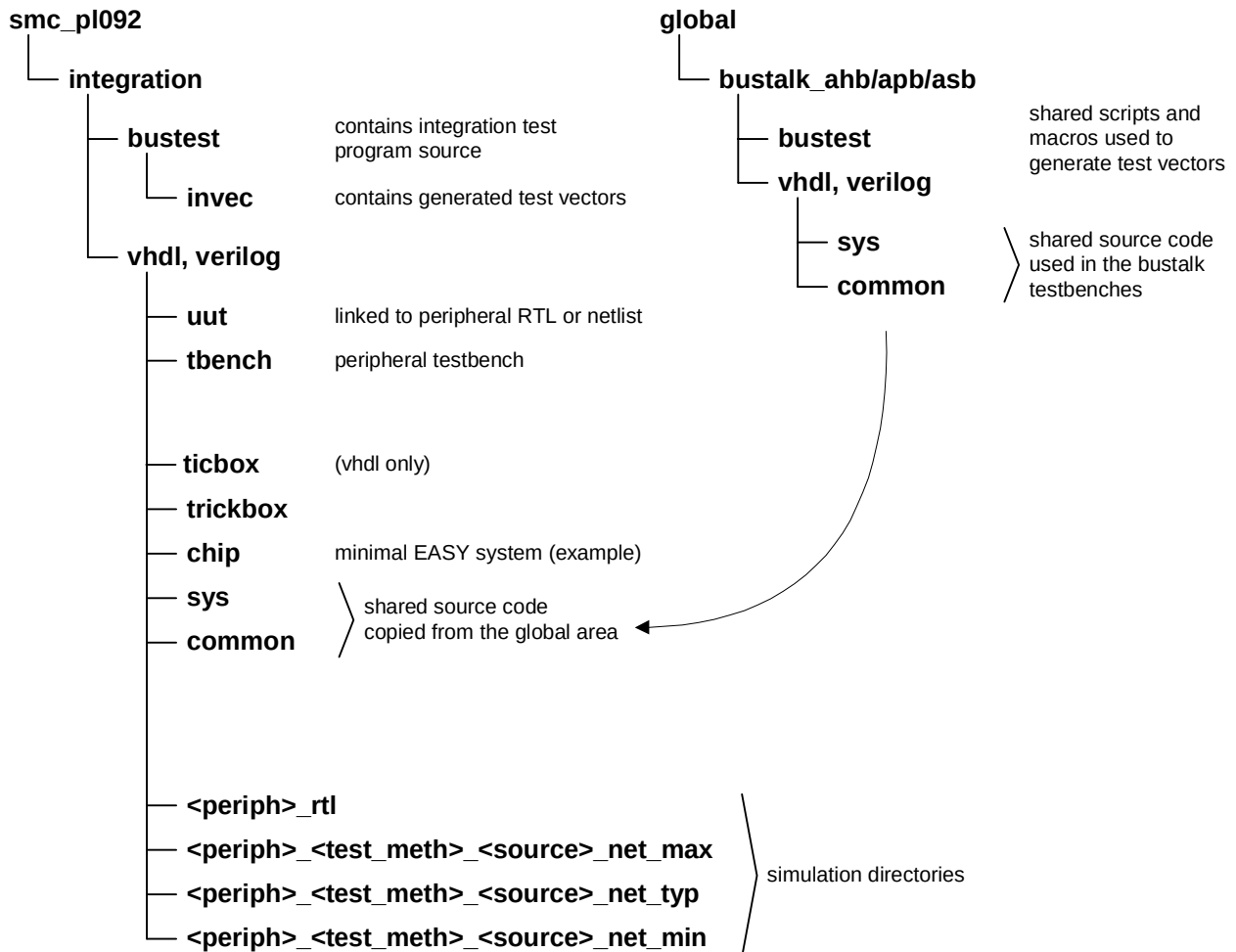


Figure 3-5. Generic TicTalk testbench directory structure

4 USAGE INSTRUCTIONS

4.1 Introduction

This PrimeCell SMC has been designed in both synthesizable VHDL and Verilog Hardware Description Languages (HDL).

The peripheral is supplied with a standalone functional verification testbench using BusTalk methodologies. Additionally, an integration testbench is supplied based on AMBA TicTalk for purposes of testing integration vectors.

4.2 Design tools used

The release note for the PrimeCell SMC describes the simulation and synthesis tools used. Where different design tools are to be used, then amendments to the configuration, script and make files will be required.

The simulation examples below are illustrated using Model Technology Inc, ModelSim commands for both VHDL and Verilog designs, and Cadence LDV for the Verilog designs. Amend these accordingly when other simulation tools are to be used.

VHDL and Verilog simulations have been performed with Model Technology Inc, ModelSim. The ModelSim simulator is an event-driven simulator which produces compiled environments for both VHDL and Verilog.

Verilog simulations have also been performed using Cadence LDV. The LDV simulator is an event-driven simulator which interprets the HDL at run-time.

Denali Tools are required for simulations. A Denali license can be obtained from the Denali website. A limited period license can be obtained for the purposes of repeating the verification supplied with the PL092. After that period of time, it is up to the customer to obtain a full license for their site.

For further information, refer to the README files within the tbench directories for each testbench.

4.3 Reference Cell library

The Avant! CB25 (0.25 μ m) standard cell library is supplied with each PrimeCell peripheral as a reference cell library.

This proprietary Avant! Passport standard cell library has been used during development of the peripheral, for synthesis and gate-level simulation with post-synthesis back-annotated timings in Standard Delay Format (SDF). For further information on this cell library, refer to the documentation supplied.

Use of a different technology library or design tools requires that script files and configuration files must be amended for the user's chosen technology library, simulation and synthesis tools.

For further information, particularly regarding amendment of synthesis script files to a different technology library, and integration of peripherals in an AMBA-based ASIC, refer to the Integration Manual of the peripheral concerned.

The availability of this reference cell library provides a means for the end user to replicate the synthesis and verification processes performed prior to delivery. Please note that this is a reference library only with no physical views, and users must provide their own target library for the development of their system.

The gate counts and performance relate to the reference library only and it is believed that this library is representative of a typical vendor cell library. Synthesis to the customers target library should yield similar gate counts, but performance is vendor library specific.

Figure 4.1 illustrates how the cell library directory structure should be created relative to the PrimeCell peripherals. The top-level cb25 directory can be placed in the same directory level as the top-level of the PrimeCell peripherals, in order to run the gate-level simulations as defined in configuration files, shell scripts and usage instructions described later.

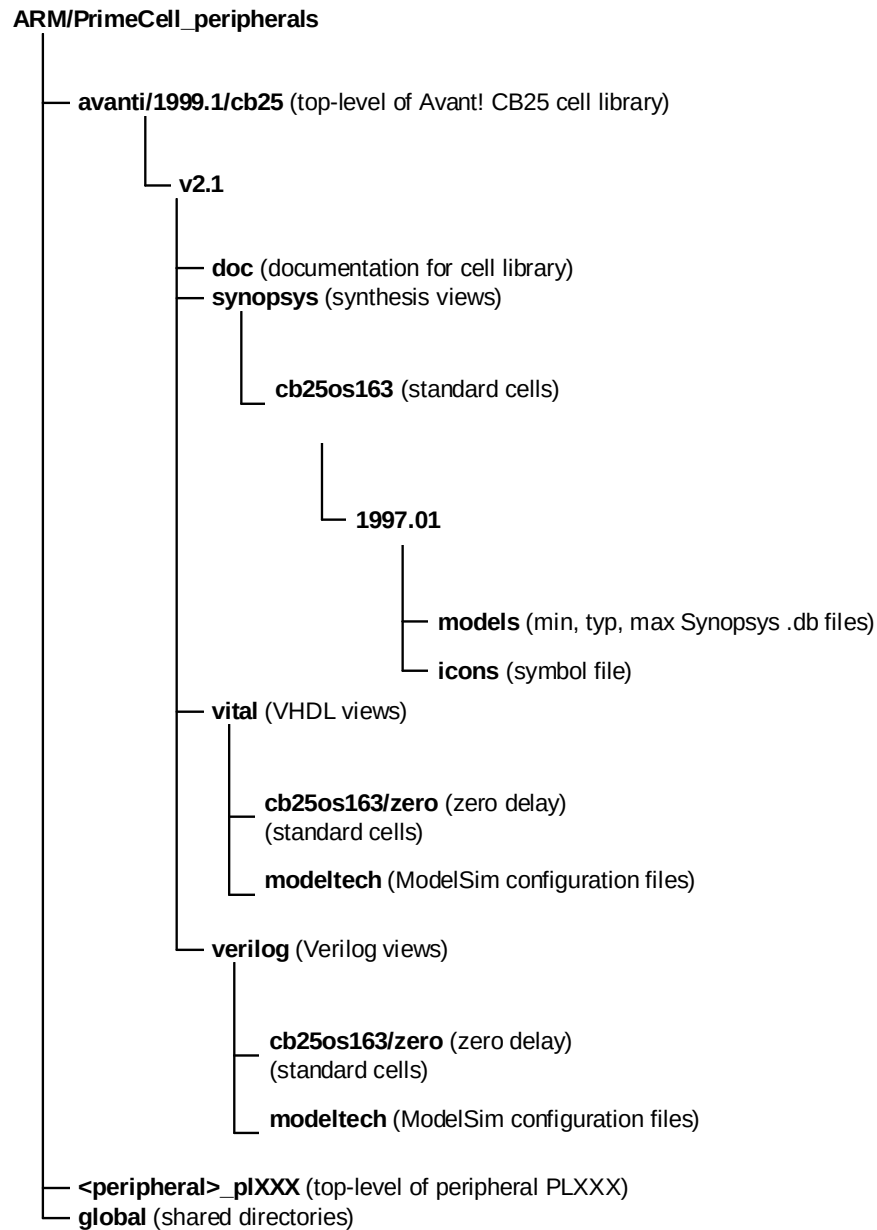


Figure 4.1 Avant! CB25 cell library directory structure

4.4 Peripheral Development Environment

The peripheral development environment is designed to support a number of standard tools within a defined directory structure. To control how these tools are used a number of UNIX environment variables and makefiles are provided. This minimises the amount of user interaction required to support the tool chain.

To support the different requirements of the end user, a number of design parameters are supported. These are used to define which of the supported design flows is to be used.

Additionally, environmental parameters are supported to allow the end user to customise the peripheral development environment, allowing integration with their own development environment.

In principle, running a design tool (simulation, synthesis, ...) consists of two steps:

- 1) setting up environment variables
- 2) using make tool to start a design tool

The make tool will then create necessary directory structure, setup all files required and start design tool.

4.4.1 Environment variables

UNIX environment variables are used to simplify the configuration of the directory paths, design tools and design flow options. A brief overview of the variables is given in **Table 4-1**.

Variable	Value	Description
GLOBAL	<global_dir_path>	The path to the shared files.
PERIPH	Smc	The peripheral name
SIMULATOR	modelsim, LDV	Defines which simulator is going to be used.
TEST_METH	noscan, scanready, scaninsert	Defines the scan methodology adopted during synthesis.
HDL_SOURCE	vhdl, verilog	The source code type – VHDL or Verilog.
HDL_NETL	vhdl, verilog	The netlist type – VHDL or Verilog.
TEST_ENV	BUSTALK, TICTALK	Defines the test environment - functional or integration test.
LIB_VHDL	<vhdl_library_path>	The location of the VHDL cell library.
DIRS_VERILOG	<verilog_library_path>	The location of the Verilog cell library.
FILES_VERILOG	<verilog_udp_path>	The location of the Verilog UDP file.
LIB_SYNOP	<synopsys_library_path>	The location of the synopsys target library.

Table 4-1 Environment Variables

The peripheral's top-level makefile will overwrite the PERIPH variable with the correct name, but before running the make routine this variable must be declared within the UNIX environment. This enables integration of the PrimeCell peripheral makefile structure into a project (ASIC) makefile structure, where multiple PrimeCell peripherals might exist and each one of them requires the PERIPH variable to have a different value.

4.4.2 Makefile commands

The makefile structure consists of the peripheral specific makefiles and the shared makefiles (located in the global directory, which is referenced by the GLOBAL environment variable). The peripheral makefiles are used to move through the peripheral directory structure, setup the environment and then execute a shared makefile. The peripheral makefiles consist of the top-level makefile located in the smc_pl092 directory and the makefiles in the peripheral sub-directories.

The shared makefiles are located in the \$GLOBAL directory, these makefiles are common to all peripherals in a system/ASIC and contain design tools specific commands. This enables an easy and quick modification to the makefile structure, if a design flow or a design tool needs to be changed.

A summary of the top-level makefile commands is given in **Table 4-2**. Only the top-level makefile will be explained here, see the "README" and "makefile" files in the peripheral sub-directories for more details.

Make Argument	Commands	Description
all	cd \$HDL_SOURCE make all	compiles RTL source code for simulation
rtl	cd verification	block level RTL simulation

	make rtl	
netlist	cd synopsys make netlist	synthesis
net_max	cd verification make net_max	maximum delay netlist simulation
net_typ	cd verification make net_typ	typical delay netlist simulation
net_min	cd verification make net_min	minimum delay netlist simulation
clean	cd vhdl; make clean cd verilog; make clean cd verification; make clean cd integration; make clean cd synopsys; make clean	removes work files in the sub-directories

Table 4-2 Makefile Commands

4.5 Running Design Tools

This chapter describes a step-by-step procedure for performing various tasks in the design flow.

4.5.1 Generating the Functional Test Vectors (BusTalk)

The BusTalk test compliance programs are written in 'C' and located in the verification/bustest directory. The generated BusTalk (.sim and .bif) test vector files are located in the verification/bustest/invec directory.

To generate the test vectors, first setup the following environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Smc
setenv SIMULATOR  modelsim, LDV
```

Then in the smc_pl092 directory (peripheral's root dir) type:

```
cd verification/bustest
make Smc1
```

This will create two test vector files in the verification/bustest/invec directory, the infile.bif is used in the VHDL testbench and the bif.sim in the Verilog testbench.

The Functional Tests are too large to be run in one go (due to the size of the tif and bif files generated), so have been split up into six sections, Smc1, Smc2, Smc3, Smc4, Smc5 and Smc6. These tests must be compiled separately and six verification simulations run, one for each set of tests. Note that tests 1 to 3, and 6 run on the standard testbench, while test 4 runs on the Denali testbench and test 5 runs on the Denali1 testbench. See the file smc_pl092/verification/bustest/README for more information on running the six sets of tests.

To run a single test in isolation:

```
cd verification/bustest
make <test_name>    e.g. make RegisterTests
```

to generate a test vector that contains only one test. See the README and makefile files in the verification/bustest directory for more details.

4.5.2 Compiling the Peripheral RTL Code

Ensure that the relevant tools have been sourced.

To compile the peripheral RTL code, first set the environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Smc
setenv SIMULATOR  modelsim, LDV          - selects ModelSim or LDV simulator
setenv HDL_SOURCE  vhdl, verilog          - selects VHDL or Verilog RTL code
setenv TEST_ENV    BUSTALK
```

Then, in the smc_pl092/\$HDL_SOURCE/rtl_source directory, type:

```
make all
```

to compile the peripheral RTL code in the correct order.

4.5.3 Running Functional Simulations

Ensure that the relevant tools have been sourced.

Before running the functional simulation, ensure that the functional test vectors are generated. To perform a functional simulation of the peripheral RTL, first set the environment variables:

```
setenv GLOBAL <global_dir_path>
setenv PERIPH Smc
setenv SIMULATOR modelsim, LDV      - selects ModelSim or LDV simulator
setenv HDL_SOURCE vhdl, verilog      - selects VHDL or Verilog RTL code
setenv TEST_ENV BUSTALK
```

Then, in the smc_pl092 directory, type:

```
make rtl
```

to run the functional simulation of the peripheral RTL code. The simulation will terminate correctly with the following message to signal the completion of the test:

** Failure: End of test

(This message is a consequence of the simulator mechanism for termination)

There should be no errors reported during the simulation for a successful run, except during the initialisation period. It is advised that simulation transcript files always be checked for any errors or warnings during any subsequent simulations. The simulation log file will be written to the verification/\$HDL_SOURCE/Smc_rtl directory.

The simulation log file name depends on the simulator used, for ModelSim it is 'transcript' and for LDV it is 'verilog.log'.

The supplied simulation log files are:

- verification/vhdl/Smc_rtl/SmcX_rtl_modelsim.log	VHDL RTL, ModelSim simulator
- verification/verilog/Smc_rtl/ SmcX_rtl_modelsim.log	Verilog RTL, ModelSim simulator
- verification/verilog/Smc_rtl/SmcX_rtl_LDV.log	Verilog RTL, LDV simulator

where 'X' is the number of the test vectors, 1-6.

Some additional steps are required when running verilog simulations. Please see smc_pl092/verification/verilog/README_Denali.

4.5.4 Running Synthesis

Ensure that the relevant tools have been sourced.

Synopsys Design Compiler is used for the peripheral synthesis, which targets the Avant! CB25 standard cell library.

The synopsys/scripts directory contains the peripheral specific synthesis scripts and the shared scripts are located in the \$GLOBAL/synopsys_shared/scr_common directory. In order to change the target library or the clock frequency, only the shared scripts will need to be changed.

To run synthesis, first set the environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Smc
setenv TEST_METH   noscan, scanready, scaninsert - selects the test method
setenv HDL_SOURCE  vhdl, verilog                - selects VHDL or Verilog RTL code
setenv HDL_NETL    vhdl, verilog                - selects VHDL or Verilog netlist
setenv LIB_SYNOP   <synopsys_lib_path>
```

To make use of the synopsys cache:

Check that the global/synopsys_shared/synopsys_cache directory exists and is write enabled.

Write enable the smc_pl092/ synopsys/log and smc_pl092/synopsys/report directories.

Then, in the smc_pl092 directory, type:

```
make netlist
```

Synthesis log and report files will be written in the synopsys/log and synopsys/report directories, respectively.

The HDL_SOURCE variable selects the HDL source being compiled and the HDL_NETL variable controls the output format of the netlist.

The TEST_METH variable selects the test methodology used for the synthesis:

- noscan: - the peripheral will be synthesised without inserting a scan chain and scan enabled flip-flops
- scanready: - the peripheral will be synthesised with scan enabled flip-flops, but without inserting a scan chain
- scaninsert: - the peripheral will be synthesised with a scan chain inserted

4.5.4.1 scr_common directory

The scr_common directory contains the scripts that are shared by the PrimeCell peripherals. The files listed below are referenced in the peripherals synthesis script and must always be present.

- setup.scr – this file is used to define various Synopsys variables that are to be used during synthesis e.g. bus naming style.
- global.scr – this file defines global synthesis parameters which must be applied to all the PrimeCell peripherals e.g. set_max_area.
- amba_params.scr – these files define the timing parameter to be used for the various types of AMBA interfaces.
- ahb_slave.scr – these files define the timing constraints to be used for the various types of AMBA interfaces.
- library_<libraryname>.scr – this file contains everything to do with the cell library and the operating conditions. Using a single file to define all the library information makes it simple for the customer to switch the target library.
- name_rules.scr – the Synopsys naming rules, used to change the names of the cells, nets and ports in the design.

4.5.4.2 scripts directory

The scripts directory is specific to the peripheral and contains the synthesis command script and the synthesis constraints file that apply only to the peripheral. This directory contains the following files ;

- Smc.cmd - this is the synthesis command script used for the peripheral. This command script references the scripts contained in this and scr_common directories.
- Smc_scan.cmd – this is the command script used specifically for scan insertion. If scan insertion is not required this command script will not be used.
- Smc.scr – this file defines the timing requirements for the peripheral's non-amba signals.
- Smc_exceptions.scr – this file contains all the timing exceptions e.g. false and multi-cycle paths.
- Smc_BusMode.scr – this file defines the tie-off value applied to the EXTBUSMUX input, optimising out the DBI or EBI logic as required. The default setting is LOW, optimising out the EBI interface logic.

Scripts also exist for the sub-blocks of the Smc, as these are synthesised individually before the top-level of the Smc is synthesised.

4.5.4.3 log directory

The log directory is used to store the information generated about the synthesis run. Two files are created:

- Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}_synth.log – this file is generated within the synthesis command script and is used to record the output from the checks performed on the design being synthesised i.e. "check_timing" and "check_test" both pre and post scan insertion.
- Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}_run.log – the synthesis run log is generated by piping the output of "dc_shell" into this file.

All errors in these files must be fixed, and the number of warning must be reduced to a minimum. The synthesis run log file might have following errors, which should be ignored:

Error: You don't have a 'HDL-Compiler' license to remove.

Error: You don't have a 'Test-Compiler' license to remove.

NOTE: When the “noscan” option is used the ‘check_test’ command isn’t executed as it requires a TestCompile licence.

4.5.4.4 report directory

The report directory contains the reports generated from the synthesis run. The file name of the generated report varies depending on the parameters defined for the synthesis run i.e. the source code type and the test requirement. This allows different configuration to be synthesised without over-writing existing report files.

The reports generated by the synthesis scripts can be split into the a number of groups; violations report, timing reports, area report, constraint report and a scan chain report:

- **Area report:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.area

The area report contains the area for each of the modules in the design.

- **Cell report:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.Cell

The cell report contains a list of all the cells in the design.

- **Clock Tree report:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.ClockTree

The clock tree report contains a list of all the cells in the design connected to each clock input. This can be used to ensure that the clock is only fed to registers, and is not gated.

- **Constraint reports:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.constr
“Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.ports

For further information the constraints used during synthesis are reported in the following two files.

The constraint report (*.constr) contains general information about the netlist that has been created, this report is generated using the “report_design”, “report_constraint”, “report_clock” and “report_attribute –design” commands.

The ports report contains (*.ports) information about the constraints placed on each of the peripheral’s top level ports and is generated using the “report_port –verbose” command.

- **Hierarchy report:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.Hier

The hierarchy report shows the hierarchy of all blocks and cells in the design.

- **Latch report:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.latch

The latch report contains information of all latches and timing loops found in the design, the report is generated using the “all_registers -level_sensitive” and “report_timing –loops” commands

- **Timing reports:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.max
“Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.min

The two timing reports are generated to enable the actual timing of the synthesised netlist to be examined.

The max path report file (*.max) contains a timing report for the ten longest under worst case conditions and is generated using the “report_timing -delay max -path full -nworst 10” command.

The min path report file (*.min) contains a timing report for the ten shortest paths under best case conditions and is generated using the “report_timing -delay min -path full -nworst 10” command.

- **Port timing reports:** “Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.inputs_max
“Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.inputs_min

```
"Smc_${TEST_METH}_${HDL_SOURCE}_${HDL_NETL}.outputs_max
"Smc_${TEST_METH}_${HDL_SOURCE}_${HDL_NETL}.outputs_min
```

The four report files contain full timing path reports for the input and output ports, showing the maximum and minimum paths to/from registers in the design.

- **Reference report:** "Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.Ref

The reference report contains a list of the cell library references used in the design, and counts of the number used in the design.

- **Reset Tree report:** "Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.ResetTree

The reset tree report contains a list of all the cells in the design connected to each reset input. This can be used to ensure that the reset is only fed to registers, and is not gated.

- **Scan chain report:** "Smc_scaninsert_\${HDL_SOURCE}_\${HDL_NETL}.scan

The scan chain report is created by the "Smc_scan.cmd" script and is only generated when scan insertion has been performed. The report details the expected fault coverage, the scan configuration and the scan chain order using the "report_test -coverage", "report_test -configuration" and "report_test -scan_path" commands respectively.

- **Violation report:** "Smc_\${TEST_METH}_\${HDL_SOURCE}_\${HDL_NETL}.vio

The violation report contains a summary of all the constraint violations, the report is generated by the "report_constraint -verbose -all_violators" command.

The only acceptable violation is a design area violation.

Similar reports are also generated for the Smc sub-blocks.

4.5.5 Running Netlist Simulations

Ensure that the relevant tools have been sourced.

Before running the netlist simulation, ensure that the functional test vector is generated. The vhdL or verilog cell library must also be compiled and performing a "make clean" in the respective area can do this:

```
cd avanti/1999.1/cb25/v2.1/vital/cb25os163/zero/      then, type> make clean
```

```
cd avanti/1999.1/cb25/v2.1/verilog/cb25os163/zero/  then, type> make clean
```

To run the netlist simulation, set the environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Smc
setenv SIMULATOR  modelsim, LDV
setenv TEST_METH   noscan, scanready, scaninsert - selects the test method
setenv HDL_SOURCE  vhdL, verilog                - selects VDHL or Verilog RTL code
setenv HDL_NETL    vhdL, verilog                - selects VHDL or Verilog netlist
setenv DIRS_VERILOG <verilog_cell_lib_path> - set if HDL_NETL = verilog
setenv LIB_VHDL    <vhdL_cell_lib_path>         - set if HDL_NETL = vhdL
```

Note that the TEST_METH, HDL_SOURCE and HDL_NETL variables should have the same values for the peripheral synthesis and the netlist simulation.

Then, in the `smc_pl092` directory, type:

```
make net_max to run 'worst case' netlist simulation
make net_typ to run 'typical case' netlist simulation
make net_min to run 'best case' netlist simulation
```

The simulation will terminate correctly with the following message to signal the completion of the test:

```
** Failure: End of test
```

There should be no errors reported during the simulation for a successful run, except during the initialisation period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations. The simulation log file will be written to one of the following directories:

<code>verification/\$HDL_NETL/Smc_<test_meth>_<hdl_source>_net_max</code>	worst case simulation log
<code>verification/\$HDL_NETL/Smc_<test_meth>_<hdl_source>_net_typ</code>	typical simulation case log
<code>verification/\$HDL_NETL/Smc_<test_meth>_<hdl_source>_net_min</code>	best simulation case log

The simulation log file name depends on the simulator used, for ModelSim it is 'transcript' and for LDV it is 'verilog.log'.

The supplied simulation log files are:

- `verification/verilog/Smc_noscan_vhdl_net_min/SmcX_noscan_vhdl_net_min_modelsim.log`
- `verification/verilog/Smc_noscan_verilog_net_max/SmcX_noscan_verilog_net_max_modelsim.log`
- `verification/verilog/Smc_scaninsert_verilog_net_typ/SmcX_scaninsert_verilog_net_typ_LDV.log`

where 'X' is the number of the test vectors, 1-5.

* Note: The group of tests Smc6 are not run on the netlists, as the default synthesis configuration optimises out the EBI interface logic, and these tests only check the EBI interface.

4.5.6 Generating the Integration Test Vectors

To generate the integration test vectors, first set the environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Smc
setenv SIMULATOR  modelsim, LDV
```

Then, in the `smc_pl092` directory type:

```
cd integration
make default
```

This will create the integration test vectors in the `smc_pl092/integration/bustest/invec` directory.

Note that the integration tests are only an example and that they should be modified when the peripheral is integrated into the system. Please see the peripheral's integration manual for more details.

4.5.7 Running the Integration Test RTL Simulation

To perform the integration test simulation, first set the environment variables:

```
setenv GLOBAL      <global_dir_path>
setenv PERIPH      Smc
setenv SIMULATOR  modelsim, LDV
setenv HDL_SOURCE  vhdl, verilog
```

Then, in the smc_pl092 directory type:

```
cd integration
make rtl
```

The simulation will terminate correctly with the following message to signal the completion of the test:

```
** Failure: End of test
```

There should be no errors reported during the simulation for a successful run, except during the initialisation period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations. The simulation log file will be written to the /integration/\$HDL_SOURCE/Smc_rtl directory.

Note that the integration testbench is only an example of an ASIC system and that the integration test should be performed on the actual system. Please see the peripheral's design manual for more details on the integration testbench.

4.5.8 Simulation error and warning messages

Note If the Smc4 set of tests is run on the standard testbench or the Denali testbench is run without the models correctly set up, multiple read errors will be generated.

4.5.8.1 VHDL

The following warning messages may be found in the simulation log files during the initialisation period and are not significant

```
# ** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand, the
result will be 'X'(es)!
```

There should be no error messages in the simulation logs.

4.5.8.2 Verilog

Modelsim

The following errors will be generated for the Smc1 test:

```
# BWERRHTRES : At reset Master is not driving HTRANS to IDLE. TIME :    9624592 ns
# BWERRHTRES : At reset Master is not driving HTRANS to IDLE. TIME :   10411444 ns
```

This is due to an inconsistency between the VHDL and Verilog testbenches.

The following cell violation will be generated for the Smc4 Denali test when run with the noscan Verilog netlist using min timing on the CB25 cell library:

```
# *Denali* Error: Memory Instance: tb_Denali.u0SRAM2 Time: 4500176990 ps
# Setup Violation (twp1=10 ns) of 9700 ps on signal webar
```

This is due to a glitch on the output multiplexer used to drive the **nSMBLS** output when accessing 8-bit memory devices. All of the inputs to the multiplexer are from clocked DFFs, so this is glitch is due to the internal paths not being balanced correctly during synthesis for minimum delay, as it is expected they will be fixed during the layout process.

Verilog-XL

The following errors will be generated for the Smc1 test:

```
# BWERRHTRES : At reset Master is not driving HTRANS to IDLE. TIME :      9624592 ns
# BWERRHTRES : At reset Master is not driving HTRANS to IDLE. TIME :     10411444 ns
```

This is due to an inconsistency between the VHDL and Verilog testbenches.

Errors similar to the following will be generated for all netlist simulations:

```
../uut/Smc_scaninsert_verilog.sdf21 L42: SDFA Error: Unable to find source port
tbench.uut.SMDATAIN[20]
```

These are due to a problem with bussed ports in SDF, and can be ignored. The `-x` option to the Verilog-XL command list will remove the error messages.

4.5.9 Use of Denali simulation models

This section describes the steps involved in using the DENALI memory models. These are used to provide accurate simulation of real memory devices, as part of the verification testing.

Denali memory models are third party memory models written by Denali, a company specialising in memory simulation models.

To be able to run the full set of verification tests, you need to download a limited period licence from the Denali web site (www.denalisoft.com).

Section 11.1.24 of the Design Manual PL092_DDES_0000_pdf, contains details of the Denali models used.

The following table shows for each test, whether to use the Denali testbench or the standard testbench.

Test	Testbench to be Used
Smc1	Standard
Smc2	Standard
Smc3	Standard
Smc4	Denali
Smc5	Denali1
Smc6	Standard
ResetTests	Standard
RegisterTests	Standard
AddrToggleTests	Standard
TransferTests	Standard
BankCrossingTests	Standard
BusAccessTests	Standard
ROMTests	Standard

Cornercase1	Denali
Cornercase2	Denali
Cornercase3	Denali
Cornercase4	Denali
Cornercase5	Standard
Cornercase6	Standard
Cornercase7	Standard
Cornercase8	Standard
Cornercase9	Standard
Cornercase10	Standard
Cornercase11	Standard
Cornercase12	Standard
Cornercase14	Standard
BROMDelayTest	Denali1
BROMTests	Standard
BROMBusyInsertionTest	Denali
RBLETests	Standard
CSTests	Standard
EndiannessTests	Standard
RemapTests	Standard
ProtectionTests	Standard
SMWAITTests	Standard
SMWAITFlagTests	Standard
DelayValueTests	Standard
MemoryBankTests	Standard
PerformanceTests	Standard
RandomTests	Standard
MCReqGntTests	Standard
DenaliTests	Denali
BigEndHburstTest	Standard
OneClkAfterReset	Standard
BROMCornerCase	Standard
WPErrorsTest	Standard
BusyinsCornerCase	Denali
MixedAccessTests	Standard
DelayValueROMTests	Standard
NewRandomTest	Standard
EbiCornerCase	Standard

Table 4-3 Testbenches to be used with the tests